Capstone Project: Final Submission & Presentation Guideline

Final Presentation	Week 16 — Wednesday, June 10, 2026 (in class)
Final Submission Deadline	Friday, June 19, 2026, 23:59
Presentation Length	30 min max per team **including demo**; demo capped at 10 min. Roughly equal speaking time across both members.
Weight	Final presentation + report + source code + demo video = **30%** of final grade (10 pts presentation + 20 pts submission bundle)
Team	Same 2-person team from the proposal

Overview

The submission has **two parts**, both required and graded together:

- **Part A — Final Presentation (Week 16, in class):** ~30-minute presentation by both team members with a demo (live or recorded).
- **Part B — Submission Bundle (uploaded by Friday June 19, 23:59):** GitHub repository + Docker image + report PDF + slides PDF + demo video.

Read both parts before you start preparing — the report and the slides reference the same architecture diagrams and accuracy results, and the CI/CD pipeline contributes to the source-code grade.

Part A — Final Presentation (Week 16, in class)

Format and Time Budget

Total time per team	30 minutes total, **including demo** (strict — the instructor will cut you off)
Demo time	Capped at **10 minutes** (live or recorded). Counts against the 30-minute total — the longer the demo, the less time for slides.
Speaking balance	Both members must speak roughly equally. A team where one member dominates the time will be marked down on the *Presentation* rubric.
Speaker switching	Hand off in **blocks**, not slide-by-slide. The default is one handoff (M1 covers the first half, M2 covers the second). More than one handoff is allowed if the content genuinely calls for it (e.g., one member co-presents a slide they led the work on), but avoid frequent switching — it disrupts flow and eats time.
Suggested split	If you use a 5-minute demo: ~20 min slides + 5 min demo + 5 min Q&A. If you use a 10-minute demo: ~15 min slides + 10 min demo + 5 min Q&A. Adjust to fit your project.
Demo	Required. Either **live** (your Jetson on the projector) or **recorded** (screen + camera composite). See "Demo Requirements" below.
Slide count	**15–20 content slides** + 2 heading/title slides ≈ **17–22 total**. Aim for ~60–75 seconds per content slide depending on how long your demo runs.

Anything in the **15–20 content-slide** band fits the budget. If you need 25+ content slides, talk to the instructor first.

The 30-minute clock includes the demo. A 10-minute demo leaves ~15 minutes for slides + ~5 minutes for Q&A. A 5-minute demo leaves ~20 minutes for slides + ~5 minutes for Q&A. Plan accordingly — and if you can squeeze in an end-to-end dry-run with a timer (Week 15 has no lab session, which makes it a natural slot), do it.

Suggested Slide Structure

The slide *order* below is a strong recommendation. The slide *count* per section is the budget — you can spend a slide more here and a slide less there, but if the totals drift far from 20 you will run over time.

#	Section	Slides	Lead
1	**Title slide** (project name, both members, advisor, date)	1	both
2	**Problem statement & motivation** — what real-world problem, who benefits, why edge AI (the same argument from your proposal, refined)	2	M1
3	**Final system architecture** — block diagram showing Sense → Process → Decide → Act, MQTT topics, Docker container boundary, what runs on which Jetson kit	2	M1
4	**What changed vs the proposal** — one slide of "we proposed X, we shipped Y, here's why"	1	M1
5	**AI model & optimization choices** — model selected and why, training/fine-tuning data, the TensorRT precision you shipped (FP16 or INT8), calibration approach if INT8	2	M1
6	**Section transition**	1 (heading)	—
7	**Implementation highlights** — 3–4 things you built that you're proud of, e.g. multi-camera sync, the actuator control loop, INT8 calibration script, the MQTT bridge	3	M2
8	**CI/CD pipeline** — your GitHub Actions workflow graph, the gates it enforces (lint, coverage ≥ 90%, security, accuracy, integration test on Jetson)	2	M2
9	**Test set & methodology** — what data you held out, how you measured accuracy, how you instrumented latency / FPS / CPU% / GPU% / power	1	M2

#	Section	Slides	Lead
10	**Performance requirements & optimization journey** — what numerical targets you set up front (FPS, latency, accuracy, max watts, CPU%/GPU% headroom); your **baseline** run before any tuning; each optimization step you applied (model choice, TensorRT precision, input resolution, frame-skipping, power-mode pinning, threading model) and what each one moved on each metric; where you ended up vs the targets	2	both — M1 leads requirements + baseline, M2 leads optimization steps + final achievement
11	**Results** — final accuracy on the held-out set; FPS and p50 / p95 / p99 latency; **CPU% and GPU% utilization (from `tegrastats`)**; power draw (the `VDD_IN` rail value `tegrastats` already reports — no extra tooling needed)	2	M2
12	**Live or recorded demo**	(off-slides, up to 10 min — counts against 30-min total)	both
13	**What we'd do differently if we did it again** — 3–5 honest items, not platitudes (the lifecycle reflection: what we'd change end-to-end if we ran the project again)	1	both — M1 + M2 each take half
14	**Acknowledgments + Q&A prompt**	1	both

That's 19 content slides + 1 title + 1 transition heading = **21 slides** (plus the off-slides demo).

Demo Requirements

The demo is graded both during the presentation (Part A) and as a turn-in artifact (Part B.7). The same recording can serve both. Length: **maximum 10 minutes** (4–7 min recommended).

The demo must clearly show the **complete Sense → Process → Decide → Act loop** running on the Jetson Orin Nano: the system senses the world via a live sensor, processes the data with an AI model, makes a decision, and **physically acts** on that decision via an actuator (buzzer, LED, servo, relay, motor — display-only output does not count). You may pre-record it if live setup at the venue is risky (Wi-Fi, lighting, projector).

What the demo must show, in order:

1. The Jetson powered on, with a visible terminal or status indicator confirming it's the Orin Nano (e.g., `tegrastats` running in a corner, or a clear physical shot of the device with the camera attached).
2. The sensor producing live input — not a pre-recorded video file, not a saved test image. If you absolutely must use a recorded video to make the demo deterministic, say so explicitly during the demo and explain why.
3. The AI inference pipeline running — bounding boxes overlaid on the live stream, FPS counter visible, and (ideally) GPU utilization or power draw shown via `tegrastats`.
4. The decision logic firing — the moment an event triggers (e.g., "person enters zone", "PPE missing", "occupancy threshold crossed"), pause and call it out.
5. The actuator responding — the buzzer beeping, the LED changing color, the servo moving, the relay clicking. **Camera must be pointed at the actuator** so it's visible in the recording, not just described.
6. A **dashboard UI** (FastAPI + WebSocket / MJPEG style introduced in Week 11) running on a laptop or browser, showing the live MJPEG stream, detection events, FPS, and any other telemetry your system publishes. The dashboard should update in real time as the demo unfolds.

Recorded demos: up to 10 minutes (4–7 min recommended), MP4 or MOV, audible audio if the actuator makes sound. Put it on a USB drive *and* include it in the submission bundle (Part B.7).

Live demos: Arrive 15 minutes early to test the projector, the network (or hotspot), and the camera. Have the recording ready as a backup — if the live demo fails after 90 seconds, switch to the recording. Teams running a live demo must still produce a recording for the submission bundle.

Q&A

~5 minutes. Both team members should be prepared to answer questions about any part of the system. Common question patterns:

- "Why did you pick model X over model Y?"
- "What happens if the camera is disconnected?"
- "How would this scale to 50 devices?"
- "What's the worst failure mode you observed during development?"
- "Walk me through what your CI does on a pull request."

Presentation Grading (10 pts of the 30-pt capstone total)

Criterion	Pts	What earns full marks
Slide quality & narrative	2	Clean visuals, no wall-of-text, architecture diagram is readable from the back of the room, 15–20 content slides on budget
Speaking balance & delivery	2	Both members speak roughly equally; neither reads slides verbatim; transitions are clean
Technical depth	2	The "why we made this choice" reasoning is visible throughout, not just "what we did"
Demo	3	Sense → Act loop clearly visible on the Jetson; actuator on camera; runs without obvious panic
Q&A	1	Both members answer questions; "I don't know" is acceptable when honest
Total	**10**	

Part B — Submission Bundle (due Friday June 19, 2026, 23:59)

The bundle is a single GitHub repository link plus a small zip of artifacts that don't fit in git. The repo and its CI history are graded directly — the instructor will `git clone`, `pdm install`, run `pytest`, look at your latest CI run, and try `docker pull` your image.

What You Submit

#	Artifact	Where	Format
1	**GitHub repository URL**	submitted via TronClass / iLearn	public repo, latest commit on `main` is what gets graded
2	**Docker image**	published to GHCR (`ghcr.io/<owner>/<project>:<tag>`)	multi-arch or arm64-only; runs on Jetson with `--runtime nvidia`

#	Artifact	Where	Format
3	**Final report PDF**	committed at `report/FINAL_REPORT.pdf`	8–12 pages, see required sections below
4	**Slides PDF**	committed at `report/PRESENTATION.pdf`	the deck you presented in class
5	**Demo video**	committed at `report/DEMO.mp4` (if < 50 MB) **or** uploaded to YouTube/Vimeo unlisted with link in `report/DEMO_LINK.md`	MP4, ≤ 10 min (4–7 recommended)
6	**Test artifacts zip**	uploaded to TronClass / iLearn as `<team>_test_artifacts.zip`	held-out test set + accuracy / utilization CSVs (see §B.8 below)

Both team members submit the same bundle to **TronClass** / **iLearn** (Datung University's course system). Source code lives in GitHub, but the official turn-in goes through TronClass / iLearn — that submission timestamp is the timestamp of record. Pushing to GitHub after the deadline does not extend it.

B.1 — GitHub Repository (Required)

One repository. `main` branch is what gets graded.

Visibility lifecycle:

1. **During development through grading:** keep the repo **public**. GitHub Actions minutes are free and unlimited on public repos, which matters because the 5-stage CI runs on every push. Public ≠ open-source — public just means anyone with the URL can read the code; it does not grant reuse rights.
2. **After Friday June 26, 2026:** flip the repo to **private** and invite the instructor and TA as collaborators (read access is sufficient). This protects your work product while preserving the grading record.
3. **Open-sourcing for portfolio / public sharing:** requires Datung University's written approval first — see the FAQ.

Repository structure (mirror HW6's layout — the rubric grades it directly):

```
<project-name>/
├── .github/
│   └── workflows/
│       ├── ci.yml          # Pull-request + push-to-main pipeline (5 stages)
│       └── deploy.yml      # Tag-triggered deploy (optional but recommended)
├── src/                    # All Python source. One main class per file.
```

```
├── tests/
│   ├── test_*.py           # Unit + property tests (≥ 90% coverage on src/)
│   └── integration/        # Tests that need a real Jetson (run via self-hosted runner)
├── deploy/                 # docker-compose.yml, deploy.sh, healthcheck.sh, rollback.sh
├── calibration/            # If you use INT8 — calibration dataset + script
├── data/                   # Held-out test set (small images only; large data in test_artifacts)
├── scripts/
│   └── parse_tegrastats.py # Parses tegrastats.log → utilization.csv
├── report/
│   ├── FINAL_REPORT.pdf
│   ├── PRESENTATION.pdf
│   └── DEMO.mp4 or DEMO_LINK.md
├── accuracy_baseline.json  # Baseline mAP / accuracy that the accuracy gate compares against
├── pyproject.toml          # PDM-managed; ruff + pytest + coverage configured
├── Dockerfile              # Builds the runtime image
├── README.md               # Setup, MQTT topic map, how to run, how to demo
└── LICENSE                 # do not add until/unless school approves open-sourcing
```

B.2 — Source-Code Compliance (Same Rules as HW6)

The capstone code is graded against the **same compliance rules** as HW6. The grader will skim every `.py` file and run `ruff check` — visible violations cost points.

File header — mandatory on every `.py`, `.sh`, and `.yml` file:

```
#!/usr/bin/env python3
# Copyright (c) 2026 <Your Name(s)>
# Datung University — I4210 AI實務專題
```

For shell scripts, `#!/usr/bin/env bash` shebang. For YAML files (workflows, compose), drop the shebang but keep the two attribution comments at the top. README.md uses an equivalent markdown header (project title + "by ... at Datung University" subtitle).

One main class per file. Each non-trivial source file in `src/` houses exactly one main class matching the filename, with helpers folded into the class as private methods. Module-level functions are reserved for true utilities. Examples:

File	Class	Module-level helpers OK
`src/inference_node.py`	`InferenceNode`	`main()` thin entry point
`src/actuator_controller.py`	`ActuatorController`	none
`src/zone_counter.py`	`ZoneCounter`	none
`src/mqtt_publisher.py`	`MqttPublisher`	none

Test files (`tests/test_*.py`) follow pytest convention: collections of `def test_*` functions, no class wrapper required. Headers still mandatory.

`pyproject.toml` : PDM-managed. The grader runs `pdm install` and expects it to succeed on a clean machine. All runtime dependencies pinned to a major version. `ruff` , `pytest` , `pytest-cov` listed as dev dependencies. The `[tool.coverage.run]` and `[tool.pytest.ini_options]` sections must be present and consistent with the CI's `--cov-fail-under` flag.

Type hints + docstrings on all public methods. Private methods (`_helper`) can skip docstrings if the name is self-explanatory.

B.3 — Docker Image (Required)

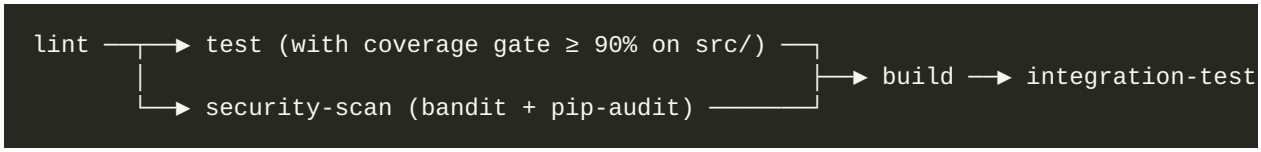
You must publish at least one Docker image to GHCR that runs your full inference pipeline on the Jetson.

Requirement	Notes
Multi-arch or arm64-only build, pushed to `ghcr.io/<owner>/<project>`	grader will `docker pull` and run
Runs on Jetson with `docker run --runtime nvidia ...`	hardware-accelerated, not CPU fallback
Accepts configuration via env vars or a mounted config file — no hardcoded paths from your laptop	reproducible across machines
Image tag matches the git tag (`v1.0.0` , `v1.0.1` , ...) and the `:latest` floating tag points at the newest stable release	Lab 12 / HW6 pattern
Compose file in `deploy/docker-compose.yml` defines the runtime, network, and volume mounts	one-command demo
`Dockerfile` defers GPU-specific work (TensorRT engine compilation) to the entrypoint, not the build	build runs in CI on a no-GPU runner

The README must include the exact `docker pull` and `docker run` commands a grader can copy-paste.

B.4 — CI/CD Pipeline (Required, 5 pts)

Your GitHub Actions workflow at `.github/workflows/ci.yml` must trigger on pull-request and push-to-main and gate at minimum the following five jobs (the same DAG as HW6 Part B):



Each job must:

Job	What it must do	Pass criteria
`lint`	`ruff check src/ tests/` on hosted Ubuntu runner	Zero violations
`test`	`pdm run pytest --cov=src --cov-fail-under=90` on hosted Ubuntu runner	≥ 90% statement coverage on `src/`; all unit tests green
`security-scan`	`bandit -r src/` + `pip-audit` on hosted Ubuntu runner	Zero high-severity findings; documented allowlist if you waive any
`build`	`docker buildx` arm64 build, push to GHCR with `:sha-<commit>` tag	Image successfully pushed
`integration-test`	Pull the image on the self-hosted Jetson runner, start the container, run `tests/integration/` against the live system	All integration tests green

Additional production patterns earn no extra points but make your demo dramatically easier:

- **Tag-triggered deploy** (`deploy.yml` on `v*` push) with a required-reviewer approval gate — exactly the HW6 Part D pattern.
- **`rollback.sh`** with a state file that recovers to the previous tag in under 30 seconds.

The grader will look at your latest workflow run on `main` and read the YAML. A green badge with all five jobs passing earns the full 5 pts.

Self-Hosted Jetson Runner (Required)

GitHub-hosted runners (Ubuntu cloud VMs) have no GPU, no CUDA, no camera, no MQTT broker, and no `nvpmodel`. Any job that needs hardware-accelerated inference, the camera, the actuator, or `tegrastats` must run on **your own Jetson registered as a GitHub Actions self-hosted runner**.

Concretely, in this capstone the following jobs must run on the self-hosted Jetson runner:

- The `integration-test` job in `ci.yml` (pulls your GHCR image on the Jetson, starts the container, exercises the camera + AI + actuator end-to-end).

- The `deploy.yml` workflow that pulls the tagged image and runs `deploy/deploy.sh` on the Jetson.
- Any benchmark / utilization-capture jobs you add (those need real `tegrastats` data).

Target jobs at the Jetson runner using the standard label set:

```yaml
jobs:
  integration-test:
    runs-on: [self-hosted, linux, arm64, jetson]
    needs: build
    steps: ...
```

Setup references (already covered in earlier weeks — re-use, don't reinvent):

- `lab/LAB_12.md` Step 5 (Stretch) — initial registration of the Jetson runner against your GitHub repo.
- `homework/HOMEWORK_6.md` Step 0.0 — re-pointing or fresh-installing the runner against a new repo (Options A / B / C); SSH plumbing for the deploy job is in HW6 Step 0.6.

If your team did not complete Lab 12 Step 5, follow HW6 Step 0.0 Option C for a fresh install on the Jetson. Either way, by the time you wire up `integration-test` in `ci.yml` the runner must already be online and showing as **Idle** in your repo's Settings → Actions → Runners page.

B.5 — Testing (Required — Similar Coverage to HW6)

Your test suite must include the same five categories HW6 grades. The **statement coverage on `src/` must be ≥ 90%**, enforced in CI by `--cov-fail-under=90`. The number of individual test functions is whatever it takes to reach that bar — no minimum count.

Category	What it tests	Where it runs
Unit tests for inference logic	Model loads, output shape, post-processing, decision logic boundaries	hosted runner
Unit tests for messaging	MQTT publish/subscribe round-trip, JSON schema, retry logic	hosted runner
Accuracy gate	Model on the held-out test set produces accuracy within N pts of the baseline (mAP for detection, accuracy for classification); reads `accuracy_baseline.json`	hosted runner
Integration tests	End-to-end on real Jetson — pull image, run for N seconds, check inference + actuator output	self-hosted Jetson runner

Category	What it tests	Where it runs
Coverage gate	Statement coverage on `src/` **≥ 90%**, configured in `pyproject.toml` and enforced by `--cov-fail-under=90` in CI	hosted runner

If your project has a hardware actuator that's hard to test in CI, mock the GPIO library at the test boundary and assert that the right call was made.

B.6 — Final Report (PDF — 8–12 pages)

Write the report in markdown and convert to PDF using the same `tools/md2pdf.sh` you used for HW6. Commit at `report/FINAL_REPORT.pdf`. Required sections, in order:

§	Section	Length	Content
1	**Project Problem Statement**	½–1 page	What problem does the system solve? Who is the user? Why does it require edge deployment? (Refine the proposal's argument with what you actually learned during implementation.)
2	**Final Architecture**	1–2 pages	Architecture diagram showing Sense → Process → Decide → Act, MQTT topic map, Docker container boundary, hardware wiring, what runs on which Jetson. **Call out what changed vs the proposal architecture and why.**
3	**Implementation Highlights**	1–2 pages	3–5 things you built that demonstrate technical depth. Code organization (one class per file, src/ layout), the TensorRT precision you shipped and the calibration approach, MQTT design, actuator control, CI/CD pipeline.
4	**Test Set Description**	½ page	What data you used to evaluate the model. Source (custom-collected, public dataset, mix), size, label distribution, how you split train/val/test, what edge cases you specifically included (e.g., bad lighting, occlusion, adversarial conditions).

§	Section	Length	Content
5	**Performance Requirements & Optimization Journey**	1–2 pages	The lifecycle reflection. State the **numerical targets** you set up front (e.g., "≥ 25 FPS, p95 latency ≤ 60 ms, mAP@50 ≥ 0.65, power ≤ 15 W under load, GPU% ≥ 70 to confirm we're using the accelerator"). Show the **baseline** numbers from your first end-to-end run before any tuning. Then walk through the **optimizations you applied** in order — model choice, TensorRT precision selection, input resolution, frame-skipping, threading model, async I/O — and the delta each one moved on each metric. End with a "did we hit the targets?" honest verdict. A small table of "step → FPS → latency → CPU% → GPU% → watts" with one row per optimization step is the strongest format here.
6	**System Performance Results**	1 page	Final numbers, presented as tables. **Accuracy:** mAP@50 for detection, top-1 / top-5 for classification, F1 / precision / recall as appropriate, all on the held-out test set. **Latency table:** FPS, p50 / p95 / p99 latency at the precision you shipped (state which precision — FP16 or INT8 — and any tradeoff you accepted, e.g., "INT8 cost us 1.8 mAP points but doubled our FPS"). **Resource & power table:** average and peak CPU% / GPU% under sustained load, plus average power draw in watts — all sampled from a single `tegrastats` run (see "How to capture utilization data" below). One power number is fine; no need to test multiple `nvpmodel` modes.
7	**Lessons Learned**	1 page	3–5 specific things you learned that you didn't know in Week 6. "Honest" beats "impressive" — what surprised you, what was harder than expected, where the documentation was wrong, where TensorRT failed silently.
8	**What We'd Do Differently if We Did It Again**	½–1 page	3–5 specific items. Not "we'd start earlier" (everyone says that). Real items: "we'd choose the precision target on day one and stick to it, not bolt it on at week 13"; "we'd write integration tests before the actuator wiring"; "we'd pick a model with a smaller backbone — the 8 GB RAM ceiling bit us at week 14".
9	**Individual Reflections**	½ page each	One per team member, **report-only** (these don't appear on slides). What you specifically built. What you learned. How the team divided work. (Same format as the homework reflections.)

§	Section	Length	Content
10	**Acknowledgments & References**	¼ page	Open-source libraries used, datasets cited, code patterns borrowed (with link). **You must cite anything you copied verbatim, including from course materials.**

Plain markdown with tables is fine. Use specific numbers throughout (e.g., "28 FPS at FP16 with 14 W power, GPU 78% / CPU 22% under sustained load, mAP@50 = 0.71 on 240 held-out images"), not vague claims.

How to Capture Utilization Data (Required for §5 + §6)

`tegrastats` is the canonical tool — it streams a line per second with CPU%, GPU%, RAM, power rails, and temperatures. Capture it during a sustained-load run of your system (≥ 60 seconds with the camera live and inference flowing) and save the raw output for the test-artifacts zip:

```
# On the Jetson, in one terminal — stream tegrastats to a file
sudo tegrastats --interval 1000 --logfile tegrastats.log

# In another terminal — start your inference container at full load
docker compose -f deploy/docker-compose.yml up

# After ≥ 60 s of steady-state inference, stop tegrastats and parse
# (a small Python script in your repo at scripts/parse_tegrastats.py is fine)
```

Convert `tegrastats.log` to a CSV (`utilization.csv`) with columns: `t, cpu_avg_pct, gpu_pct, ram_used_mb, vdd_in_mw, vdd_cpu_mw, vdd_gpu_mw, vdd_soc_mw, gpu_temp_c, cpu_temp_c`. Compute mean / p50 / p95 / max per column for the report tables. Commit the parser script and check the CSV into the test-artifacts zip.

B.7 — Demo Video (Required)

Up to 10 minutes (4–7 min recommended), MP4 or MOV. **Same demo as Part A** — one recording serves both the in-class presentation and the submission bundle. Commit at `report/DEMO.mp4` if under 50 MB; otherwise upload to YouTube/Vimeo as **unlisted** and put the link in `report/DEMO_LINK.md`. Public uploads OK if you want them on a portfolio, but unlisted is fine for grading.

The video must show the **complete Sense → Process → Decide → Act loop** on the Jetson — live sensor in, AI inference visible (bounding boxes / FPS counter), decision firing, and the physical actuator visibly responding (buzzer audible, LED changing color, servo moving, relay clicking). **Record it before presentation day.**

B.8 — Test Artifacts Zip (Uploaded to TronClass / iLearn)

Anything that doesn't fit in git lives here:

- The held-out test set (images + labels) used for the accuracy results in §6
- Raw `tegrastats.log` from at least one ≥ 60-second sustained-load run, plus the `utilization.csv` you parsed from it (columns: `t, cpu_avg_pct, gpu_pct, ram_used_mb, vdd_in_mw, vdd_cpu_mw, vdd_gpu_mw, vdd_soc_mw, gpu_temp_c, cpu_temp_c`)
- Raw CSV results from your latency / FPS benchmarks (one row per inference, with timestamps)
- Optimization-journey log: a small CSV or markdown table with one row per optimization step from your report §5 ("baseline → small model → INT8 → 416×416 → MODE_15W") so the grader can verify the journey numbers
- Calibration dataset (if you shipped INT8)
- Any large model weights not tracked in git (use Git LFS for these instead if practical)

Zip name: `<team-name>_test_artifacts.zip`. The grader spot-checks one or two teams' results by re-running their accuracy script and `parse_tegrastats.py` against this data — make sure both scripts read from a path you can override via env var or CLI flag, so the grader can point them at the unzipped folder.

Submission Bundle Grading (20 pts of the 30-pt capstone total)

Component	Pts	What earns full marks
GitHub repo structure & file headers	2	Layout matches the spec; every `.py` / `.sh` / `.yml` has the required header
Source-code compliance (one class per file, type hints, docstrings, ruff clean)	3	`ruff check` exits zero; class layout consistent; public methods documented
Docker image runs on Jetson	2	`docker pull` + `docker run --runtime nvidia` succeeds for the grader without modification
CI/CD pipeline	5	All 5 stages exist; latest run on `main` is green; coverage gate ≥ 90%; security scan present

Component	Pts	What earns full marks
Testing breadth & coverage	3	All 5 test categories present; statement coverage on `src/` ≥ 90%; accuracy gate green; integration test green on real Jetson
Final report (PDF)	3	All 10 sections present; §5 optimization journey shows real before/after numbers per step; §6 results include CPU% / GPU% utilization from `tegrastats`; honest lessons learned
Demo video	1	Sense → Act loop clearly visible on the Jetson, audible if relevant
Test artifacts zip	1	Held-out data + result CSVs included; accuracy script can be re-run by grader
Total	**20**	

Combined with Part A (presentation, 10 pts), the capstone submission is 30 pts of your final grade.

Timeline (Week 12 → Final Submission)

HW6 runs in parallel — handed out W12, due before W14 — and builds the same CI / deploy / rollback patterns the capstone reuses, so it's two-for-one work. Weeks 12–13 are the most time-pressured of the term.

Week	Capstone work	HW6 (parallel)	Output
Before W12 (per proposal W9–11)	Proposal approved (W9). Custom dataset collected if needed + baseline model trained or chosen (W10). TensorRT FP16 engine running on Jetson; MQTT pipeline skeleton wired (W11).	— (HW6 not started yet)	All proposal milestones cleared before Week 12 begins.
W12	Continue **core feature implementation** per proposal (detection / tracking on live camera). Read this guideline. Audit repo against §B.1 layout. Stand up `ci.yml` skeleton (lint + test on hosted runner).	Read spec; register self-hosted Jetson runner (HW6 Step 0.0); start Part A.	Capstone repo public; CI skeleton green; HW6 runner online.

Week	Capstone work	HW6 (parallel)	Output
W13	**Integration + secondary features** per proposal — full Sense→Process→Decide→Act loop wired, actuator firing, Week 11 dashboard UI connected. Containerize: `Dockerfile` + `deploy/docker-compose.yml`.	Main HW6 work — 5-stage graph, security-scan, integration test, tag-deploy, rollback. **Heaviest week.**	Sense→Act loop demo-able locally; Docker image builds.
W14	**HW6 due early in week.** Port HW6 patterns into the capstone repo: full 5-stage CI on Jetson runner (re-pointed at capstone), push Docker image to GHCR with semver tags. Write held-out test set + accuracy gate (`accuracy_baseline.json`). Run `tegrastats` → `utilization.csv` . First end-to-end accuracy / latency / FPS / CPU% / GPU% / power numbers. Draft report §1–§4.	HW6 due.	Capstone CI fully green incl. integration-test on Jetson; image on GHCR; first measured numbers; report §1–§4 drafted.
W15	(Lecture only — no lab.) Run **optimization journey** iterations (baseline → each step → final), capturing per-step FPS / latency / CPU% / GPU% / power. Finalize dashboard UI. Finish report §5–§10. Build 15–20 content-slide deck. **Record demo video** (≤ 10 min). Optional dry-run.	—	All artifacts at draft quality: report PDF, slides PDF, demo MP4, test artifacts zip.
Wed 6/10	**Final presentation in class** (30 min, demo ≤ 10 min).	—	Live presentation + demo.
Wed 6/10 → Fri 6/19	Apply feedback. Final commits (CI green). Re-run capture if numbers shifted. Regenerate PDFs. **Submit via TronClass / iLearn** — GitHub URL + test artifacts zip.	—	TronClass / iLearn submission complete.
After Fri 6/26	Flip GitHub repo to **private**. Invite instructor + TA as collaborators. (See §B.1.)	—	Repo private; instructor + TA can still access.

FAQ

Q: Is CI/CD required? Yes. The CI/CD pipeline is graded directly (5 pts).

Q: Our actuator is hard to test in CI. What do we do? Mock the GPIO library at the test boundary and assert the right call was made with the right arguments. The integration test on the real Jetson runner verifies the actuator actually fires.

Q: Can we use a private repo? The grading repo must be **public** through the grading window (so GitHub Actions stays free and unlimited). After grading, flip the repo to **private** and invite the instructor and TA as collaborators with read access. See B.1 — Visibility lifecycle.

Q: Public repo means our project is open-source, right? No. Public means "readable by anyone with the URL" — it does not grant reuse rights. Open-source requires an explicit license (MIT, Apache-2.0, etc.) on top of being public. Do not add an open-source license without school approval; see *"Can we open-source the project after the course?"* below.

Q: What if the live demo fails during the presentation? Switch to the recorded demo video. If the live demo fails after 90 seconds, switch.

Q: Can one team member do most of the speaking? No. Both members must speak roughly equally — this is graded. If one member has a documented disability that affects speaking, talk to the instructor in advance.

Q: Our final scope differs from the Week 9 proposal. Is that OK? Yes — document the change in §2 of the report ("what changed vs the proposal"). Silently shipping a different project than what was approved is not OK.

Q: Can we open-source the project after the course? Follow Datung University's IP / open-source policy. Do not add an open-source license, publish to a portfolio site, or share the code beyond the grading audience until you have obtained written approval from the school. After grading the repo should be flipped to private with the instructor and TA invited as collaborators.

Q: Do we have to use ROS2? No. The capstone has no ROS2 requirement. You may use ROS2 if you prefer it for your architecture.

Q: Do we have to deploy to multiple Jetsons? No. A single Jetson is sufficient. If your project uses both kits, the report architecture diagram should make clear which kit runs which component.

Where to Get Help

- **HW6 reference:** `homework/HOMEWORK_6.md` in this same week — file headers, 5-stage CI workflow, coverage gate config, rollback pattern. The capstone uses the same standards.
- **Lab 12 reference:** `lab/LAB_12.md` — GHCR push and self-hosted runner setup.
- **Spec clarifications and architecture questions:** contact the instructor via Line. We can set up a meeting time as needed.
- **Hardware troubleshooting and debugging help:** make an appointment with the TA on campus at Datung.

Good luck.